CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT
School of Engineering and Technology (SOET), Bhubaneswar

FORMULA 1 TYRE STRATEGY ANALYSIS USING PYTHON

Comprehensive Viva Voce Master Guide & End-to-End System Documentation

Subject Code: CUTM1018 — Data Analytics
Department: Computer Science & Engineering (B.Tech CSE)
Supervisor: Sabyasachi Mohanty

Presented By:
Pratik Raj — Reg. No: **250301120218**
Swayam Suraj Nayak — Reg. No: **250301120227**
Debashish Pradhan — Reg. No: **250301120238**

DOCUMENT PURPOSE & VIVA DEFENCE OBJECTIVES

This document provides full viva voce preparation for the B.Tech CSE Data Analytics project. It contains: (1) A complete architectural and code audit of every file in the project repository; (2) Elevator pitch scripts (30s, 1m, 3m, 5m); (3) Presentation speaking notes matching the slide deck; and (4) Over 50 categorized viva questions and answers covering Python, Pandas, FastF1, Data Preprocessing, Statistics, F1 Domain Concepts, and Examiner Trap Questions.

SECTION 1: COMPLETE FILE-BY-FILE TECHNICAL BREAKDOWN

Below is an exhaustive explanation of every file and folder in the project repository. Use this section when an examiner points to a specific file and asks: *'What does this file do and why did you build it this way?'*

• app.py — *Interactive Streamlit Web Dashboard (User Interface Layer)*

Entrypoint of the web application. Implements a responsive dark-themed dashboard with 6 analytical tabs: (1) Tyre Strategy & Stints, (2) Degradation & Thermal Cliff, (3) Pit Stop Execution, (4) Lap Pace & Head-to-Head, (5) Automated Key Insights, and (6) Data Audit. Utilizes `@st.cache_data` for sub-second reactive state updates and incorporates an automatic offline fail-safe that loads cached session payloads if Wi-Fi drops during a presentation.

• requirements.txt — *Package Dependency Manifest*

Defines pinned, reproducible library dependencies: FastF1 (v3.8+), Pandas (v2.2+), NumPy (v1.26+), Plotly (v5.20+), Streamlit (v1.35+), SciPy (v1.12+), Scikit-Learn (v1.4+), and Pytest (v8.0+). Guarantees environment reproducibility.

• README.md — *Master Project Documentation*

Comprehensive technical documentation covering project overview, problem statement, technology stack justification, folder structure, CLI installation steps, empirical findings from the 2023 Bahrain GP, and team member details.

• PROJECT_REPORT.md — *Formal Academic Project Thesis*

B.Tech university report structured in standard academic format: Abstract, Introduction, Problem Statement, Objectives, Dataset Description, Data Cleaning Pipeline, Algorithms & Equations, Results, Causation vs Correlation Discussion, Limitations, and References.

• VIVA_AND_PRESENTATION_GUIDE.md — *Master Viva Voce & Presentation Script*

Consolidated preparation guide containing 30-second to 5-minute elevator pitches, slide-by-slide scripts for all 12 PPT slides, a live demonstration sequence, and 50+ categorized viva questions.

• .gitignore — *Version Control Exclusion Rules*

Prevents transient caches (`data/cache/`), huge binary files (`*.pkl`), virtual environments (`venv/`), and bytecode caches (`__pycache__/`) from being committed to Git.

• src/__init__.py & tests/__init__.py — *Package Initializers*

Enables Python to treat `src` and `tests` as discoverable packages, allowing clean absolute imports like `from src.data_loader import ...`.

• src/data_loader.py — *FastF1 Telemetry Ingestion & Caching Engine*

Interfaces directly with the official FastF1 API. Implements `setup_fastf1_cache()`, `load_race_session()`, and `extract_session_payload()`. Configures local SQLite and binary caching to prevent repetitive API downloads. Sets `telemetry=False` to speed up session loading by 90% while extracting lap records, tyre compound logs, and classification tables.

• src/data_cleaning.py — *Data Cleansing & Non-Destructive Filtering Pipeline*

Transforms `Timedelta` objects to continuous float seconds (`LapTime_Seconds`). Implements explicit boolean diagnostic flags: `IsPitInLap`, `IsPitOutLap`, `IsStartLap`, `IsGreenFlag`, `IsDeletedLap`, and `IsRepresentativePace`. Ensures zero silent row deletion so pit stop boundaries are preserved for pit analytics while pure flying laps are isolated for tyre wear modeling.

• src/stint_analysis.py — *Run-Length Stint Detection & Strategy Formatter*

Identifies unbroken stints on a single tyre set using change-point detection: `(IsPitOutLap | CompoundShift).cumsum() + 1`. Calculates stint boundaries (StartLap, EndLap, StintLength), average pace on clean laps, and synthesizes strategy shorthand strings (e.g. `S → S → H (2-Stop)`).

• src/tyre_analysis.py — *Tyre Compound Lifespan & Performance Analytics*

Quantifies compound metrics across Soft, Medium, Hard, Intermediate, and Wet. Calculates mileage distribution (% of race distance), stint durability statistics (mean, median, min, max stint length), and representative compound pace deltas relative to the fastest tyre.

• src/degradation_analysis.py — *OLS Regression & Fuel-Corrected Degradation Engine*

Models the rate of lap-time drop-off against tyre age using Ordinary Least Squares (OLS) linear regression. Implements the fuel correction formulation (`beta_true = beta_observed + 0.038 s/lap`), calculates statistical significance (R^2, p-value), and computes the Pace Cliff Delta (`t_late - t_early`).

• src/pitstop_analysis.py — *Pit Stop Detection & Transit Duration Analytics*

Extracts pit stop events, entry laps, compound transitions (`SOFT → HARD`), pit lane transit durations (`PitOutTime - PitInTime`), and track positions lost. Categorizes drivers into 1-Stop, 2-Stop, and 3-Stop strategies.

• src/lap_analysis.py — *Lap-Time Performance & Consistency Engine*

Computes driver pace summaries (mean, median, fastest valid lap), the Coefficient of Variation (`CV% = std / mean * 100`) to measure consistency, rolling 3-lap moving medians to smooth traffic interference, and head-to-head driver pace comparison deltas.

• src/strategy_analysis.py — *Strategy vs. Classified Position & Correlation*

Evaluates relationships between strategic choices and classified finishing positions. Computes Spearman rank correlation (`rho, p-value`) and establishes the academic distinction between Correlation, Association, and Causation.

• src/visualization.py — *High-Fidelity Plotly Data Visualization Engine*

Constructs 8 interactive Plotly charts adhering to official Pirelli colors (Soft: Red, Medium: Yellow, Hard: White, Inters: Green, Wet: Blue) and custom dark theme styling. Includes empty-data fault tolerance via `_create_empty_figure()`.

• tests/test_analysis.py — *Automated Pytest Quality Assurance Suite*

Contains 6 automated test cases covering timedelta conversions, non-destructive cleaning flags, stint detection logic, pit stop extraction, fuel-corrected degradation math, and empty DataFrame fault tolerance. Executes with 100% pass rate.

• notebooks/exploratory_analysis.ipynb — *Academic Research & Demonstration Notebook*

Step-by-step Jupyter Notebook detailing the research workflow, exploratory data analysis, and intermediate DataFrames for live examiner verification.

• data/raw/2023_bahrain_race.pkl — *Offline Telemetry Payload Cache*

Serialized snapshot of all 1,056 laps and classifications from the 2023 Bahrain GP. Enables the dashboard to run completely offline without Wi-Fi.

• assets/strategy_timeline_preview.html — *Standalone Interactive HTML Chart*

Self-contained interactive Plotly export of the full-grid Gantt stint timeline that can be opened in any web browser without running Python.

SECTION 2: ELEVATOR PITCHES (PROJECT EXPLANATIONS)

Choose your explanation depending on how much time the examiner gives you to speak:

30-Second Speed Pitch

Our project is an end-to-end Formula 1 telemetry analytics system that evaluates how tyre compound selection, stint duration, and pit stop timing influence race finishing positions. Using FastF1, Pandas, and SciPy, we built a non-destructive cleaning pipeline that removes pit lane and safety car distortions, models fuel-corrected tyre degradation slopes using linear regression, and visualizes the complete strategic progression in an interactive Streamlit web dashboard. We proved on the 2023 Bahrain Grand Prix that Soft tyres degraded at 1.77 times the rate of Hard tyres, and that a 2-stop strategy was optimal across the entire top 8 finishers.

1-Minute Standard Pitch

Good morning, respected examiners. In modern Formula 1, race strategy is just as decisive as vehicle aerodynamics and driving skill. Our project evaluates real-world telemetry from official FIA timing feeds. Raw timing contains massive anomalies—pit in-laps and safety cars add 20 to 60 seconds that falsely distort tyre wear. We built a data engineering pipeline that tags these non-representative laps without losing data, identifies tyre stints using change-point detection algorithms, and computes true tyre degradation by decoupling vehicle fuel burn-off from tyre grip loss. We deployed our analytical engine as an interactive Streamlit dashboard featuring Gantt-style stint timelines, compound usage donut charts, pit stop timelines, and head-to-head driver pace comparisons. Our findings demonstrate that while pit strategy is strongly associated with points finishes, qualifying grid position remains the primary determinant of race outcomes.

3-Minute Comprehensive Presentation

Respected supervisor and examiners, we present our Data Analytics project titled 'Formula 1 Tyre Strategy Analysis Using Python'. Under FIA regulations, every dry Grand Prix requires drivers to use at least two distinct tyre compounds. The fundamental engineering trade-off is speed versus durability: softer compounds offer high peak grip but degrade rapidly, while harder compounds offer lifespan at the cost of lap pace. Our system architecture consists of five modular tiers: (1) Data Ingestion via FastF1 with local caching; (2) Data Cleaning using boolean flags to preserve pit events while isolating clean flying laps; (3) Stint Analytics using change-point grouping to detect stints and format strategy strings like $S \rightarrow S \rightarrow H$; (4) Degradation Modeling using OLS regression with fuel burn correction (0.038 s/lap); and (5) An interactive Streamlit dashboard. In our 2023 Bahrain GP benchmark, we found that Hard tyres accounted for 55.8% of race mileage while Mediums were avoided due to graining. Soft tyres degraded at 0.133 s/lap—nearly double the 0.075 s/lap rate of Hard tyres. 100% of the top 8 finishers executed 2-stop strategies, and statistical correlation proved that qualifying grid position remains a much stronger determinant of final finish than pit stop count alone.

SECTION 3: 50+ MASTER VIVA QUESTIONS & ANSWERS

Organized into 10 academic and engineering categories. Master these short, confident answers:

CATEGORY A: PROJECT BASICS & CONTEXT

Q1: What is the title and primary objective of your project?

Answer: Formula 1 Tyre Strategy Analysis Using Python. The objective is to analyze historical telemetry to quantify the relationship between tyre compounds, stint lengths, tyre degradation, pit stops, and race finishing positions.

Q2: Who are the team members and course details?

Answer: Pratik Raj (250301120218), Swayam Suraj Nayak (250301120227), and Debashish Pradhan (250301120238) from B.Tech CSE, Centurion University, under course CUTM1018 - Data Analytics supervised by Sabyasachi Mohanty sir.

Q3: What dataset did you use, and is it real data?

Answer: We used official Formula 1 live-timing and telemetry data accessed via the FastF1 Python library, benchmarked on the 2023 Bahrain Grand Prix. It is 100% real data—no fabricated or synthetic values.

Q4: How does the application run locally?

Answer: It runs as an interactive web dashboard using Streamlit (`python -m streamlit run app.py`) backed by modular Python source code in `src/`.

Q5: What is the real-world engineering significance of this project?

Answer: It demonstrates how data analytics can deconstruct high-velocity telemetry, isolate confounding variables like fuel mass loss, and optimize strategic decision-making in competitive motorsport.

CATEGORY B: PYTHON PROGRAMMING & SOFTWARE DESIGN

Q6: Why did you choose Python for this project?

Answer: Python is the industry standard in data science and telemetry engineering due to its rich ecosystem: FastF1 for telemetry, Pandas for tabular manipulation, SciPy for regression, and Plotly for visualization.

Q7: What is the purpose of `__init__.py` in your directories?

Answer: It marks directories as Python packages, enabling clean modular imports across directories using dot notation (`from src.data_loader import ...`).

Q8: Why did you use type hints throughout the source code?

Answer: Type hints (e.g. `def clean_race_laps(df: pd.DataFrame) -> Tuple[...]`) declare expected input and return types, improving code readability, IDE static analysis, and bug prevention.

Q9: What is the difference between a module and a package in Python?

Answer: A module is a single Python `.py` file. A package is a directory containing multiple modules and an `__init__.py` file.

Q10: How does your project handle runtime exceptions?

Answer: We define custom exceptions like `F1DataError` and use structured try/except blocks to catch network drops and missing telemetry gracefully without crashing.

CATEGORY C: PANDAS & DATA WRANGLING

Q11: Why use Pandas instead of standard Python lists or dictionaries?

Answer: Pandas executes vectorized operations in compiled C/Cython, making filtering, grouping, and aggregations across thousands of lap records 50 to 100 times faster than standard Python loops.

Q12: Why did you convert lap times to seconds?

Answer: FastF1 lap times are stored as `Timedelta` objects (e.g. `0 days 00:01:38.006`). Statistical models, linear regressions, and box plots require continuous scalar floats (e.g. `98.006 s`).

Q13: What does `groupby()` do in your stint detection code?

Answer: It splits the DataFrame into subsets based on driver and assigned stint IDs, computes stint pace metrics (mean, median, min, max), and combines the results into a summary table.

Q14: What is the difference between `mean()` and `median()` lap time?

Answer: Mean is the arithmetic average, which is skewed by single slow laps (traffic or yellow flags). Median is the 50th percentile, which represents true representative race pace.

Q15: What is the difference between `.loc` and `.iloc` in Pandas?

Answer: `.loc` selects rows and columns by label or boolean mask, whereas `.iloc` selects by integer index position.

CATEGORY D: FASTF1 API & TELEMETRY INGESTION

Q16: What is FastF1?

Answer: FastF1 is an open-source Python library that provides access to official Formula 1 live timing, lap times, sector splits, tyre compound allocations, and car telemetry.

Q17: Why is local disk caching mandatory when using FastF1?

Answer: A single F1 race session contains 20 to 50 MB of telemetry. Caching stores parsed binary and SQLite data locally so repeat queries load in milliseconds without hitting API rate limits.

Q18: Why did you set `telemetry=False` in `session.load()`?

Answer: `telemetry=True` loads high-frequency 100 Hz sensor data (speed, RPM, throttle) that consumes 500 MB RAM. Setting it to `False` speeds up session loading by 90% while retaining all lap times, compounds, and pit stops.

Q19: What is `session.results` in FastF1?

Answer: It is a Pandas DataFrame containing official classification details: driver numbers, names, constructors, grid positions, finish positions, and championship points.

Q20: How does FastF1 distinguish between fresh and used tyres?

Answer: Through the `TyreLife` and `FreshTyre` attributes. `TyreLife` tracks the cumulative number of laps the physical tyre set has completed across the entire weekend.

CATEGORY E: DATA PREPROCESSING & CLEANING

Q21: Why is naive row deletion (`df.dropna()`) dangerous in F1 telemetry?

Answer: Deleting rows destroys the contiguous lap index series, breaking stint tracking and preventing pit-stop duration calculations. We use non-destructive boolean flags instead.

Q22: Why must pit in-laps and out-laps be excluded from tyre degradation analysis?

Answer: Drivers crawl through the pit lane at 80 km/h on in-laps and exit on cold tyres on out-laps, adding 15 to 25 seconds. Including them causes linear regression to show false catastrophic tyre wear.

Q23: Why exclude Lap 1 from pace modeling?

Answer: Lap 1 starts from zero speed on a stationary grid with a pack accordion effect, making it 5 to 8 seconds slower than normal flying laps.

Q24: How do you handle Safety Car laps?

Answer: We check `TrackStatus == '1'`. Any lap under Safety Car, VSC, or Red Flag (`TrackStatus` 4, 6, or 5) is flagged as non-representative because speeds are mandated by FIA delta times.

Q25: How does your IQR outlier filter work?

Answer: For each compound, it calculates the Interquartile Range ($IQR = Q3 - Q1$) and flags laps outside $[Q1 - 1.5\cdot IQR, Q3 + 1.5\cdot IQR]$ to catch unflagged spins or off-track moments.

CATEGORY F: DATA VISUALIZATION & PLOTLY

Q26: Why use Plotly over Matplotlib for the interactive dashboard?

Answer: Plotly renders interactive WebGL/SVG graphics that allow users to hover over individual laps, zoom into pit windows, and toggle traces in the legend directly in the browser.

Q27: What are the standard Pirelli tyre compound colors used in your project?

Answer: Soft is Red (`#FF1801`), Medium is Yellow (`#F5D033`), Hard is White/Silver (`#E0E0E0`), Intermediate is Green (`#39B54A`), and Wet is Blue (`#00A0DE`).

Q28: How did you implement the horizontal Stint Timeline?

Answer: Using stacked horizontal Plotly bar charts with the `base` property set to `StartLap - 1` and length set to `StintLength`, sorted chronologically by finishing position.

Q29: Why is the y-axis reversed on the Strategy vs. Position chart?

Answer: In motorsport, P1 is 1st place (the winner) and P20 is last. Reversing the axis places the winner and podium finishers at the top, matching human intuition.

Q30: What happens if a chart receives empty data?

Answer: It calls `_create_empty_figure()`, rendering an informative centered message ('Data unavailable for this selection') instead of crashing or showing a blank box.

CATEGORY G: STATISTICS & MATHEMATICAL MODELING

Q31: How did you calculate tyre degradation?

Answer: We fitted an Ordinary Least Squares (OLS) linear regression model of clean lap times against tyre age: $\text{LapTime} = \alpha + \beta_{\text{deg}} \cdot \text{TyreAge}$. The slope β_{deg} represents seconds lost per lap.

Q32: What is the fuel burn correction, and why is it necessary?

Answer: Cars burn ~1.7 kg of fuel per lap, gaining ~0.038 s/lap in lap time. We add this correction to the observed slope ($\beta_{\text{true}} = \beta_{\text{observed}} + 0.038$) so that vehicle weight loss doesn't hide tyre wear.

Q33: What is the Coefficient of Variation ($CV\%$) in your lap analysis?

Answer: $CV\% = (\sigma / \mu) \times 100$. It normalizes lap time standard deviation against the mean to quantify a driver's lap-to-lap consistency in clean air.

Q34: What is the Pace Cliff Delta?

Answer: It is the difference between late-stint pace (final 4 clean laps) and early-stint pace (first 4 clean laps): $\Delta t_{\text{cliff}} = \bar{t}_{\text{late}} - \bar{t}_{\text{early}}$.

Q35: Why use Spearman rank correlation instead of Pearson correlation for finishing positions?

Answer: Finishing positions ($P1–P20$) are ordinal ranks, not continuous normally distributed values. Spearman correlation assesses monotonic relationships on ranked variables.

CATEGORY H: FORMULA 1 DOMAIN CONCEPTS

Q36: What is a stint in Formula 1?

Answer: A continuous period of consecutive laps completed by a driver on a single set of tyres without entering the pit lane.

Q37: What is the Undercut in pit strategy?

Answer: Pitting 1 to 2 laps earlier than a rival ahead to gain fresh tyre grip, setting a fast out-lap to overtake them when they pit.

Q38: What is the Overcut in pit strategy?

Answer: Staying out longer than a rival. It works when tyre degradation is low or when the rival re-enters into heavy midfield traffic.

Q39: What is tyre graining?

Answer: When surface rubber tears away in small ripples due to lateral sliding on a cold or abrasive surface, reducing grip until the tyre surface wears smooth again.

Q40: What is dirty air?

Answer: Turbulent aerodynamic wake coming off a leading car that reduces downforce and increases tyre sliding for the chasing car.

CATEGORY I: TYRE STRATEGY FINDINGS

Q41: What was the winning strategy in the 2023 Bahrain Grand Prix?

Answer: Max Verstappen won using a 2-stop Soft → Soft → Hard (`S → S → H`) strategy.

Q42: Why was the Medium tyre barely used in Bahrain 2023?

Answer: Bahrain's abrasive track caused severe thermal graining on the Medium compound. Teams discovered in practice that Mediums degraded faster than Softs without providing Hard-tyre durability.

Q43: Which compound had the highest fuel-corrected degradation rate?

Answer: The Soft compound, degrading at 0.1334 s/lap, compared to Hard at 0.0753 s/lap.

Q44: How many pit stops did the top 8 finishers complete in Bahrain 2023?

Answer: All top 8 finishers executed 2-stop strategies.

Q45: What was the strategy of Fernando Alonso in finishing P3?

Answer: Soft → Hard → Hard (`S → H → H`), completing two long, consistent Hard tyre stints.

CATEGORY J: CRITICAL EXAMINER TRAP QUESTIONS

Q46: Does Soft always mean faster lap times?

Answer: In terms of peak chemical grip, yes. But in a race, fuel mass is a major confounder: a Hard tyre run on Lap 45 with 20 kg of fuel will often produce faster lap times than a Soft tyre run on Lap 3 with 100 kg of fuel.

Q47: Does fewer pit stops automatically mean a better strategy?

Answer: No. A 1-stop saves a 22-second pit lane loss, but if tyre wear is high (e.g. 0.13 s/lap), running 30 laps on degraded tyres loses far more cumulative time than the pit stop penalty.

Q48: Can you scientifically prove that tyre strategy caused a driver to win?

Answer: No. Strategy is an associated factor, but not the sole cause. Underlying car aerodynamic efficiency, qualifying grid position, engine power, and driver skill are massive confounding variables.

Q49: What happens if your application is run without an internet connection?

Answer: Our data loader includes an automatic offline fail-safe that intercepts connection drops and loads pre-cached session snapshots from `data/raw/*.pkl` seamlessly.

Q50: What are the primary limitations of your model?

Answer: 1. Fuel mass consumption is estimated using vehicle physics (~0.038 s/lap) rather than live fuel flow sensors. 2. Traffic wake is smoothed using rolling medians rather than 3D LiDAR/GPS proximity data.

Q51: How could Machine Learning improve this project in the future?

Answer: By training Random Forest or XGBoost models on historical weather and sector telemetry to predict optimal pit stop laps under live Safety Car probabilities.